



ДИСЦИПЛИНА	<u>Разработка приложений на языке Котлин</u> (полное наименование дисциплины без сокращений)
ИНСТИТУТ	<u>информационных технологий</u>
КАФЕДРА	<u>информационных технологий в атомной энергетике</u> полное наименование кафедры
ВИД УЧЕБНОГО МАТЕРИАЛА	<u>Лекция</u> (в соответствии с пп 1-11)
ПРЕПОДАВАТЕЛЬ	<u>Золотухин Святослав Александрович</u> (фамилия, имя, отчество)
СЕМЕСТР	<u>5 семестр 2025 – 2026 учебный год</u> (указать семестр обучения, учебный год)



Лекция 4. Разработка приложений на языке Котлин

ФИО преподавателя: Золотухин Святослав Александрович

e-mail: zolotuhin@mirea.ru



ООП. Наследование

базовый класс (класс-родитель, родительский класс, суперкласс)

производный класс (класс-наследник, подкласс)

open class базовый_класс

class производный_класс:

первичный_конструктор_базового_класса



ООП. Наследование

Пример наследования

```
open class Person{  
    var name: String = "Undefined"  
    fun printName(){  
        println(name)  
    }  
}  
  
class Employee: Person()
```



ООП. Наследование. Вызов конструктора

Первый способ:

```
class Employee: Person()
```

Второй способ

```
open class Person{  
    var name: String = "Без имени"  
    fun printName(){  
        println(name)  
    }  
}
```

```
class Employee: Person{  
    constructor() : super(){  
    }  
}
```



ООП. Наследование

```
fun main() {  
    val peter: Employee = Employee()  
    peter.name = "Петя"  
    peter.printName()  
}  
  
open class Person{  
    var name: String = "Без имени"  
    fun printName(){  
        println(name)  
    }  
}  
  
class Employee: Person()
```



ООП. Наследование класса с первичным конструктором

```
open class Person(val name: String){  
    fun printName(){  
        println(name)  
    }  
}  
  
class Employee(empName: String): Person(empName)
```



Наследование класса с первичным конструктором

```
open class Person(val name: String){  
    fun printName(){  
        println(name)  
    }  
}  
  
class Employee: Person{  
    constructor(empName: String) : super(empName){}  
}
```




ООП. Наследование

```
fun main() {  
    val peter = Employee("Петя")  
    peter.printName()  
}
```

```
open class Person(val name: String){  
    fun printName(){  
        println(name)  
    }  
}
```

```
class Employee(empName: String): Person(empName)
```



ООП. Наследование

```
fun main() {  
    val peter = Employee("Петя")  
    peter.printName()  
}  
  
open class Person{  
    val name: String  
    constructor(userName: String){  
        name = userName  
    }  
    fun printName(){  
        println(name)  
    }  
}  
  
class Employee(empName: String): Person(empName)
```



Расширение базового класса

```
fun main() {  
    val peter = Employee("Петя", "МИРЭА")  
    peter.printName()  
    peter.printCompany()  
}  
  
open class Person(val name: String){  
    fun printName(){  
        println(name)  
    }  
}  
  
class Employee(empName: String, val company: String): Person(empName){  
    fun printCompany(){  
        println(company)  
    }  
}
```



Переопределение методов и свойств

Переопределение свойств

```
open class Person(val name: String){  
    open var age: Int = 1  
}  
  
open class Person(val name: String, open var age: Int = 1){  
}  
  
open class Employee(name: String): Person(name){  
    override var age: Int = 18  
}
```



Переопределение методов и свойств

```
open class Person(val name: String, open var age: Int = 1){  
}
```

```
open class Employee(name: String, override var age: Int = 18):  
  Person(name, age){}
```



ООП. Первичный конструктор

```
fun main() {  
    val andrew = Person("Андрей")  
    println("Имя: ${andrew.name} Возраст: ${andrew.age}")  
    val peter = Employee("Петя")  
    println("Имя: ${peter.name} Возраст: ${peter.age}")  
}  
  
open class Person(val name: String, open var age: Int = 1)  
open class Employee(name: String, override var age: Int = 18):  
    Person(name, age)
```



Переопределение геттеров и сеттеров

```
open class Person(val name: String){
    open val fullInfo: String
        get() = "Человек $name - $age"
    open var age: Int = 1
        set(value){
            if(value > 0 && value < 110)
                field = value
        }
}

open class Employee(name: String): Person(name){
    override val fullInfo: String
        get() = "Сотрудник $name - $age"

    override var age: Int = 18
        set(value){
            if(value > 17 && value < 110)
                field = value
            }
}
```



Переопределение геттеров и сеттеров

```
fun main() {  
    val andrew = Person("Андрей")  
    andrew.age = 14  
    println(tom.fullInfo)  
  
    val peter = Employee("Петья")  
    peter.age = 14  
    println(peter.fullInfo)  
}
```




Переопределение методов

```
open class Person(val name: String){  
    open fun display(){  
        println("Имя: $name")  
    }  
}
```

```
class Employee(name: String, val company: String): Person(name){  
    override fun display() {  
        println("Имя: $name    Компания: $company")  
    }  
}
```



Переопределение методов

```
fun main() {  
    val andrew = Person("Андрей")  
    andrew.display()    // Имя: Андрей  
  
    val peter = Employee("Петя", "МИРЭА")  
    peter.display()    // Имя: Петя  Компания: МИРЭА  
}
```



Переопределение в иерархии наследования классов

```
open class Person(val name: String){
    open fun display(){
        println("Имя: $name")
    }
}

open class Employee(name: String, val company: String): Person(name){
    override fun display() {
        println("Имя: $name    Компания: $company")
    }
}

class Programmer(name: String, company: String):Employee(name, company){
    override fun display() {
        println("Имя: $name    Компания: $company    Должность: Программист")
    }
}
```



Запрет переопределения

```
open class Person(val name: String){
    open fun display(){
        println("Имя: $name")
    }
}

open class Employee(name: String, val company: String): Person(name){
    final override fun display() {
        println("Имя: $name   Компания: $company")
    }
}

class Programmer(name: String, company: String):Employee(name, company){
    // теперь функцию нельзя переопределить
    /*override fun display() {
        println("Имя: $name Компания: $company Должность: Programmer")
    }*/
}
```



Обращение к реализации из базового класса

```
open class Person(val name: String){
```

```
    open val fullInfo: String
```

```
        get() = "Имя: $name"
```

```
    open fun display(){
```

```
        println("Имя: $name")
```

```
    }
```

```
}
```

```
open class Employee(name: String, val company: String):  
    Person(name){
```

```
        override val fullInfo: String
```

```
            get() = "${super.fullInfo} Компания: $company"
```

```
        final override fun display() {
```

```
            super.display()
```

```
            println("Компания: $company")
```

```
        }
```

```
    }
```



Абстрактные классы и методы

Такой класс можно только унаследовать:

```
abstract class Human(val name: String){  
    fun hello(){  
        println("Мое имя: $name")  
    }  
}  
  
class Person(name: String): Human(name)
```



Абстрактные классы и методы

Стоит отметить, что в данном случае перед абстрактным классом не надо указывать аннотацию **open**, как при наследовании неабстрактных классов.

```
fun main() {  
    val jane: Person = Person("Женя")  
    val antony: Human = Person("Антон")  
    jane.hello() // Мое имя Женя  
    antory.hello() // Мое имя Антон  
}
```



Абстрактные классы и методы

```
abstract class Human(val name: String){  
    abstract var age: Int  
    abstract fun hello()  
}  
  
class Person(name: String): Human(name){  
    override var age : Int = 1  
    override fun hello(){  
        println("Мое имя $name")  
    }  
}
```




Абстрактные классы и методы

```
abstract class Human(val name: String){  
    abstract var age: Int  
    abstract fun hello()  
}  
  
class Person(name: String, override var age : Int): Human(name){  
    override fun hello(){  
        println("Мое имя $name")  
    }  
}
```



Абстрактные классы и методы

// абстрактный класс фигуры

abstract class Figure {

// абстрактный метод для получения периметра

abstract fun perimeter(): Float

// абстрактный метод для получения площади

abstract fun area(): Float

}

// производный класс прямоугольника

class Rectangle(val width: Float, val height: Float) : Figure()

{

// переопределение получения периметра

override fun perimeter(): Float{

return width * 2 + height * 2;

}

// переопределение получения площади

override fun area(): Float{

return width * height;

}



Data-классы

data class Person(val name: String, val age: Int)

При компиляции такого класса компилятор автоматически добавляет в класс функции с определенной реализацией, которая учитывает свойства класса, которые определены в первичном конструкторе:

equals(): сравнивает два объекта на равенство

hashCode(): возвращает хеш-код объекта

toString(): возвращает строковое представление объекта

copy(): копирует данные объекта в другой объект



Data-классы

```
fun main() {  
    val kate: Person = Person("Катя", 24)  
    println(kate.toString())  
}
```

```
class Person(val name: String, val age: Int)
```

Результатом программы будет следующий вывод:

Person@6d03e736



Data-классы

```
data class Person(val name: String, val age: Int)
```

Результат будет следующим:

```
Person(name=Катя, age=24)
```

```
data class Person(val name: String, val age: Int){  
    override fun toString(): String {  
        return "Имя: $name  Возраст: $age"  
    }  
}
```



Декомпозиция data-классов

```
fun main() {  
    val kate: Person = Person("Катя", 24)  
  
    val (username, userage) = kate  
    println("Имя: $username  Возраст: $userage") // Имя: Катя  
    Возраст: 24  
}  
  
data class Person(var name: String, var age: Int)
```



Перечисления Enums

Enums или перечисления представляют тип данных, который позволяет определить набор логически СВЯЗАННЫХ КОНСТАНТ

```
enum class Day {  
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY  
}  
  
fun main() {  
    val day: Day = Day.FRIDAY  
    println(day)        // FRIDAY  
    println(Day.MONDAY)  // MONDAY  
}
```



Перечисления Enums

```
enum class Day(val value: Int){  
    MONDAY(1), TUESDAY(2), WEDNESDAY(3),  
    THURSDAY(4), FRIDAY(5), SATURDAY(6), SUNDAY(100)  
}
```

```
fun main() {  
    val day: Day = Day.FRIDAY  
    println(day.value)    // 5  
    println(Day.MONDAY.value) // 1  
}
```




Перечисления Enums

```
enum class Day(val value: Int){  
    MONDAY(1), TUESDAY(2), WEDNESDAY(3), THURSDAY(4), FRIDAY(5),  
    SATURDAY(6),  
    SUNDAY(7);  
    fun getDuration(day: Day): Int{  
        return value - day.value;  
    }  
}  
  
fun main() {  
    val day1: Day = Day.FRIDAY  
    val day2: Day = Day.MONDAY  
    println(day1.getDuration(day2))    // 4
```



Перечисления Enums. Встроенные свойства и вспомогательные методы

Перечисления обладают двумя встроенными свойствами:

- **name:** возвращает название константы в виде строки
- **ordinal:** возвращает порядковый номер константы

```
enum class Day(val value: Int){  
    MONDAY(1), TUESDAY(2), WEDNESDAY(3),  
    THURSDAY(4), FRIDAY(5), SATURDAY(6),  
    SUNDAY(7)  
}  
  
fun main() {  
    val day1: Day = Day.FRIDAY  
    println(day1.name)    // FRIDAY  
    println(day1.ordinal) // 4  
}
```



Перечисления Enums. Встроенные свойства и вспомогательные методы

Кроме того, в Kotlin нам доступны вспомогательные функции:

- **valueOf(value: String):** возвращает объект перечисления по названию константы
- **values():** возвращает массив констант текущего перечисления

```
fun main() {  
    for(day in Day.values())  
        println(day)  
  
    println(Day.valueOf("FRIDAY"))  
}
```



Перечисления Enums. Хранение состояния

```
fun main() {  
    println(operate(5, 6, Operation.ADD))    // 11  
    println(operate(5, 6, Operation.SUBTRACT)) // -1  
    println(operate(5, 6, Operation.MULTIPLY)) // 30  
}  
  
enum class Operation{  
    ADD, SUBTRACT, MULTIPLY  
}  
  
fun operate(n1: Int, n2: Int, op: Operation): Int{  
    when(op){  
        Operation.ADD -> return n1 + n2  
        Operation.SUBTRACT -> return n1 - n2  
        Operation.MULTIPLY -> return n1 *n2  
    }  
}
```



Спасибо за внимание!